

Nome: Luciano da Silva Nogueira

RA: H641CC6

1. O que caracteriza uma classe e um objeto no contexto de POO?

Uma classe é como um modelo que define as características e ações de uma coisa. O objetivo é algo criado a partir desse modelo.

2. Explique o conceito de encapsulamento e qual é a convenção utilizada em Python para indicar que um atributo ou método deve ser tratado como privado.

Encapsulamento é uma forma de proteger os dados internos de uma classe e controlar como eles serão acessados. Em Python, usamos o `_` antes do nome de um atributo ou método para indicar que ele deve ser tratado como privado ou interno.

3. Como funciona o princípio da herança entre classes e de que forma uma subclasse pode estender o comportamento de uma superclasse?

Herança permite que uma classe aproveite características e métodos de outra classe, muitas das vezes a classe que herda algo para outra classe é chamado de classe mãe. Além disso, a classe filha pode adicionar novos comportamentos ou sobrescrever métodos existentes.

4. O que é o polimorfismo e como ele se manifesta na relação entre subclasses e superclasses?

Polimorfismo é quando duas ou mais classes podem ter o mesmo método, mas cada executa esse método de uma forma distinta. Um exemplo bem simples é que podemos ter duas classes, uma chamada cachorro e outra gato, as duas podem ter o método `som()` mas seu comportamento vai ser diferente, porque uma vai usar como comportamento o “latir” e outra vai usar o “miau”.

5. Qual é a função do primeiro argumento (geralmente chamado de self) em um método de instância em Python?

O `self` representa o próprio objetivo que está usando aquele método. Com ele nós podemos acessar atributos e até mesmo métodos pertencentes aquele objeto.

6. Explique a diferença prática e conceitual entre um método de instância, um método estático (`@staticmethod`) e um método de classe.

O método da instância trabalha com um objeto específico. O método estático não precisa de um objeto e o método de classe trabalha com a própria classe.

7. O que são atributos de dados de classe (class data attributes) e como eles diferem dos atributos de instância?

Atributos de classe são atributos definidos dentro de uma classe e podem ser compartilhados entre diversos objetos. Agora atributo de instância são atributos que criamos individualmente para cada objeto.

8. No contexto do padrão de projeto Template Method, o que são métodos "hook" e como eles ajudam a separar o processamento genérico de regras específicas?

Métodos Hook são métodos que permitem que uma parte específica de um processo seja aletrada ou personalizada pelas subclasses.

9. De acordo com a definição de Clean Code apresentada por Robert C. Martin (Uncle Bob), qual é o objetivo principal dessa filosofia de desenvolvimento?

O objetivo é deixar o código fácil de entender, manter e modificar, não apenas fazer ele funcionar.

10. Como o inventor do C++, Bjarne Stroustrup, descreve a relação entre um código limpo, eficiência e a quantidade de tarefas que uma rotina deve executar?

Código limpo organiza as responsabilidades, a quantidade de tarefas deve ser adequada ao propósito da rotina, e a eficiência busca realizar essas tarefas de maneira necessária e otimizada.

11. Ao definir nomes significativos no código, quais são as três perguntas fundamentais que o nome escolhido deve responder?

O nome deve deixar claro o que aquilo representar, por que existe e como está sendo usado.

12. Explique por que, segundo as boas práticas de Clean Code, os desenvolvedores não devem ter medo de utilizar nomes grandes para variáveis ou métodos.

O nome grande é essencial para deixar claro oque aquela variável ou função representa, ao menos que esse nome grande não seja objetivo e claro.

13. Com base na convenção apresentada para a Programação Orientada a Objetos, como devem ser representadas as classes e como devem ser representados os métodos?

Em Python, normalmente usamos letras maiúsculas para separar as palavras nas classes e letras minúsculas com _ para métodos. A convenção mais comum em Python utiliza PascalCase para classes e snake_case para funções e métodos.

14. O slide menciona que "métodos com muitos níveis implicam em muita responsabilidade". Como o princípio de "fazer uma única coisa" ajuda a resolver esse problema?

O princípio de responsabilidade única reduz a complexidade ao separar diferentes responsabilidades em métodos específicos. Isso torna o código mais fácil de entender, testar e alterar.

15. O que é um Code Smell e de que forma um código ruim pode impactar o andamento de um projeto e a carreira do desenvolvedor a longo prazo?

Code Smell é uma característica do código que pode indicar problemas de design, manutenção ou organização. Ele não significa necessariamente que existe um erro, mas indica uma possível necessidade de refatoração.

16. Explique o conceito do princípio DRY (Don't Repeat Yourself) e qual prática comum entre programadores ele tenta evitar.

DRY significa "Don't Repeat Yourself" e significa evitar repetir a mesma lógica ou código em vários lugares do sistema.

17. Por que o uso excessivo de comentários é visto com ressalvas no Clean Code e qual é a alternativa recomendada para tornar o código auto documentado?

Comentários são úteis, mas quando existem muitos podem mostrar que o código não está claro. É melhor tentar escrever um código que seja fácil de entender sozinho.

18. Quais são as cinco regras descritas pelo acrônimo FIRST para a criação de testes limpos?

FIRST significa que os testes devem ser rápidos, independentes, repetíveis, capazes de indicar sozinhos se passaram e escritos no momento adequado.

FIRST representa:

F - Fast: testes rápidos;

I - Independent: independentes;

R - Repeatable: repetíveis;

S - Self-Validating: capazes de validar automaticamente o resultado;

T - Timely: escritos no momento adequado.

19. Como o Princípio da Responsabilidade Única (SRP) define o motivo para uma classe ser modificada e como a falta dele afeta a coesão na POO?

SRP significa que uma classe deve ter uma responsabilidade principal e não deve fazer várias coisas diferentes ao mesmo tempo. A falta dela afeta a clareza, manutenção do código entre diversos outros problemas, porque sem a SRP podemos acumular várias responsabilidades em uma só classe ao invés de apenas criar outra classe pra ter aquela responsabilidade que foi acumulado, assim podemos destrinchar essas funcionalidades.

20. O Princípio do Aberto/Fechado (OCP) afirma que entidades devem estar "abertas para extensão, mas fechadas para modificação". Como a herança e o polimorfismo da POO ajudam a aplicar esse conceito?

O princípio diz que devemos conseguir adicionar novos comportamentos ao sistema sem precisar ficar modificando o código que já funciona.

21. Explique o Princípio de Substituição de Liskov (LSP) e o problema de abstração que ocorre quando uma classe derivada necessita de "baterias" (recursos extras não previstos na classe base).

O LSP diz que uma classe filha deve poder ser usada no lugar da classe pai sem causar problemas no funcionamento do programa. O problema de abstração ocorre quando a classe derivada precisa de recursos extras, ou “baterias”, que não foram previstos na classe base, fazendo com que ela não consiga cumprir corretamente o comportamento esperado pela classe pai.

22. Considerando que o Clean Code foca na clareza de variáveis, métodos e classes, de que maneira os conceitos do SOLID apresentados (como o SRP e o DIP) servem como base para a Arquitetura Limpa na organização dos módulos, regras de negócio e dependências externas de um sistema?

O Clean Code busca deixar o código claro e fácil de entender, enquanto o SOLID ajuda a organizar sua estrutura. O SRP separa as responsabilidades, mantendo cada módulo focado em uma função, e o DIP faz com que as regras de negócio não dependam diretamente de recursos externos. Assim, a Arquitetura Limpa organiza o sistema em módulos bem definidos, mantendo as regras de negócio independentes de detalhes externos.